

SHETH L.U.J & SIR MV COLLEGE

Aim: Implement Iterative deep depth first search for Romanian map problem.

Code: Implement A* search algorithm for Romanian map problem.

```
dict_hn = {  
    'Andheri Station': 10,  
    'DN Nagar': 7,  
    'Western Express Highway': 6,  
    'Juhu Circle': 4,  
    'JVPD': 2,  
    'PVR Dynamix Juhu Mall': 0  
}  
dict_gn = dict(  
    **{  
        'Andheri Station': dict(  
            **{  
                'DN Nagar': 3,  
                'Western Express Highway': 4  
            }  
        ),  
        'DN Nagar': dict(  
            **{  
                'Juhu Circle': 4  
            }  
        ),  
        'Western Express Highway': dict(  
            **{  
                'Juhu Circle': 5,  
                'JVPD': 6  
            }  
        ),  
        'Juhu Circle': dict(  
            **{  
                'JVPD': 2,  
                'PVR Dynamix Juhu Mall': 5  
            }  
        ),  
    }
```

Purvi.R.Karkera

T086

Subject: Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

```
'JVPD': dict(
    **{
        'PVR Dynamix Juhu Mall': 2
    }
),
'PVR Dynamix Juhu Mall': dict()
}
)
import queue as Q
start = 'Andheri Station'
goal = 'PVR Dynamix Juhu Mall'
result = ""
def get_fn(citystr):
    cities = citystr.split(" , ")
    hn = gn = 0

    for ctr in range(0, len(cities) - 1):
        gn = gn + dict_gn[cities[ctr]][cities[ctr + 1]]

    hn = dict_hn[cities[len(cities) - 1]]

    return (hn + gn)
def expand(cityq):
    global result

    tot, citystr, thiscity = cityq.get()

    if thiscity == goal:
        result = citystr + " : " + str(tot)
        return

    for cty in dict_gn[thiscity]:
        cityq.put(
            (
                get_fn(citystr + " , " + cty),
                citystr + " , " + cty,
                cty
            )
        )
    expand(cityq)
def main():
    cityq = Q.PriorityQueue()
```

Purvi.R.Karkera

T086

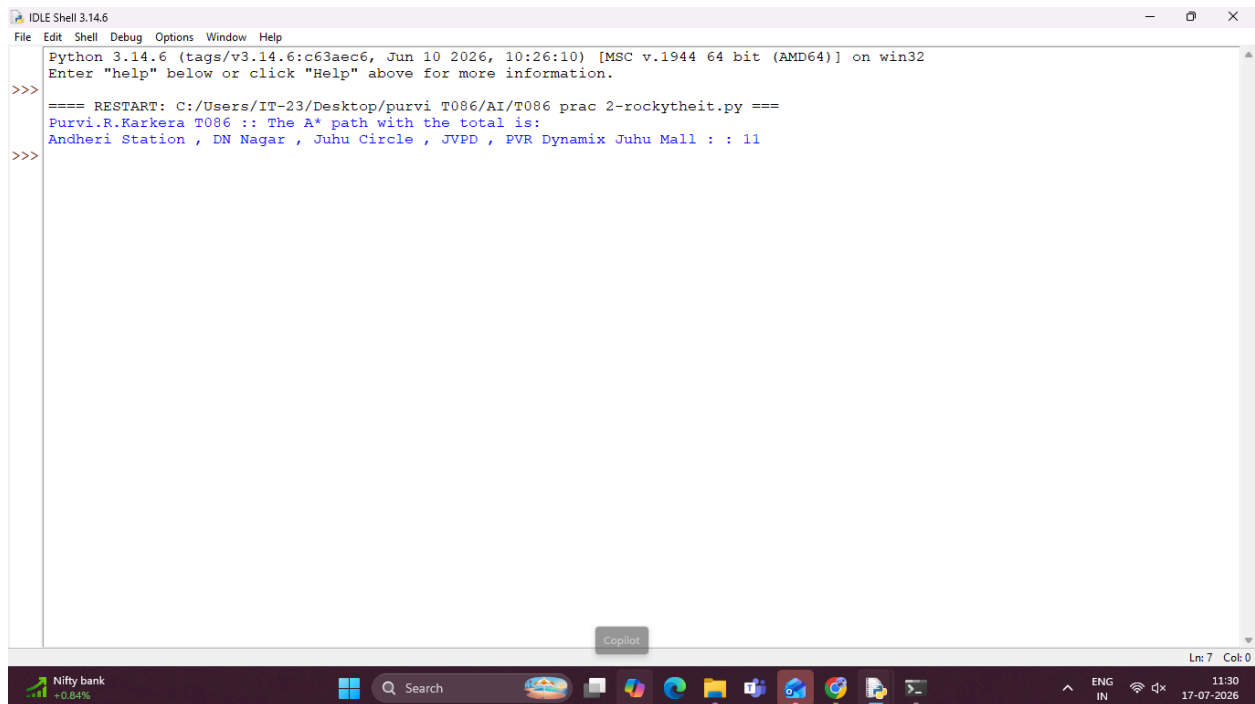
Subject:Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

```
thiscity = start

cityq.put(
    (
        get_fn(start),
        start,
        thiscity
    )
)
expand(cityq)
print("Purvi.R.Karkera T086 :: The A* path with the total is:")
print(result)
main()
```

Output:



```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/IT-23/Desktop/purvi T086/AI/T086 prac 2-rockytheit.py ====
Purvi.R.Karkera T086 :: The A* path with the total is:
Andheri Station , DN Nagar , Juhu Circle , JVFD , PVR Dynamix Juhu Mall : : 11
>>>
```

Purvi.R.Karkera
T086
Subject:Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

Code: A* Search algorithm for solving a pathfinding problem.

```
import heapq
import matplotlib.pyplot as plt
import networkx as nx
graph = {
    'Andheri Station': {'DN Nagar': 3, 'Western Express Highway': 4},
    'DN Nagar': {'Juhu Circle': 4},
    'Western Express Highway': {'Juhu Circle': 5, 'JVPD': 6},
    'Juhu Circle': {'JVPD': 2, 'PVR Dynamix Juhu Mall': 5},
    'JVPD': {'PVR Dynamix Juhu Mall': 2},
    'PVR Dynamix Juhu Mall': {}
}
heuristics = {
    'Andheri Station': 10,
    'DN Nagar': 7,
    'Western Express Highway': 6,
    'Juhu Circle': 4,
    'JVPD': 2,
    'PVR Dynamix Juhu Mall': 0
}
node_positions = {
    'Andheri Station': (2, 8),
    'DN Nagar': (1, 6),
    'Western Express Highway': (3, 6),
    'Juhu Circle': (1, 4),
    'JVPD': (3, 4),
    'PVR Dynamix Juhu Mall': (2, 1)
}
def a_star_search(graph, heuristics, start, goal):
    priority_queue = [(heuristics[start], start, [start], 0)]
    visited = set()
    while priority_queue:
        f_score, current, path, g_score = heapq.heappop(priority_queue)
        if current in visited:
            continue
        visited.add(current)
        if current == goal:
            return path, g_score
        for neighbor, edge_weight in graph[current].items():
            if neighbor not in visited:
                next_g = g_score + edge_weight
```

Purvi.R.Karkera

T086

Subject: Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

```
        next_f = next_g + heuristics[neighbor]
        heapq.heappush(
            priority_queue,
            (next_f, neighbor, path + [neighbor], next_g)
        )
    return None, float('inf')
optimal_path, total_distance = a_star_search(
    graph,
    heuristics,
    'Andheri Station',
    'PVR Dynamix Juhu Mall'
)
print(f"Purvi.R.Karkera T086 :: Optimal Path Discovered: {'' -> ''.join(optimal_path)}")
print(f"Total Road Distance: {total_distance} km\n")
G = nx.DiGraph()
for node, neighbors in graph.items():
    for neighbor, weight in neighbors.items():
        G.add_edge(node, neighbor, weight=weight)
plt.figure(figsize=(10, 8))
path_edges = list(zip(optimal_path, optimal_path[1:]))
normal_edges = [edge for edge in G.edges() if edge not in path_edges]
nx.draw_networkx_nodes(
    G,
    node_positions,
    node_size=2500,
    node_color='lightblue'
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    width=1.5,
    edge_color='gray',
    arrows=True
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    width=3.5,
```

Purvi.R.Karkera

T086

Subject: Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

```
edge_color='darkorange',
arrows=True
)

node_labels = {
    node: f"{node}\nh(n)={heuristics[node]}"
    for node in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels=node_labels,
    font_size=9,
    font_weight='bold'
)

edge_labels = nx.get_edge_attributes(G, 'weight')

formatted_edge_labels = {
    edge: f"{weight} km"
    for edge, weight in edge_labels.items()
}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=formatted_edge_labels,
    font_color='red'
)

plt.title(
    "Purvi.R.Karkera T086 :: A* Routing Map: Andheri Station to PVR Dynamix Juhu Mall\n"
    "(Highlighted Orange Path = Optimal Route)",
    fontsize=14
)

plt.axis('off')
plt.tight_layout()
plt.show()
```

Purvi.R.Karkera

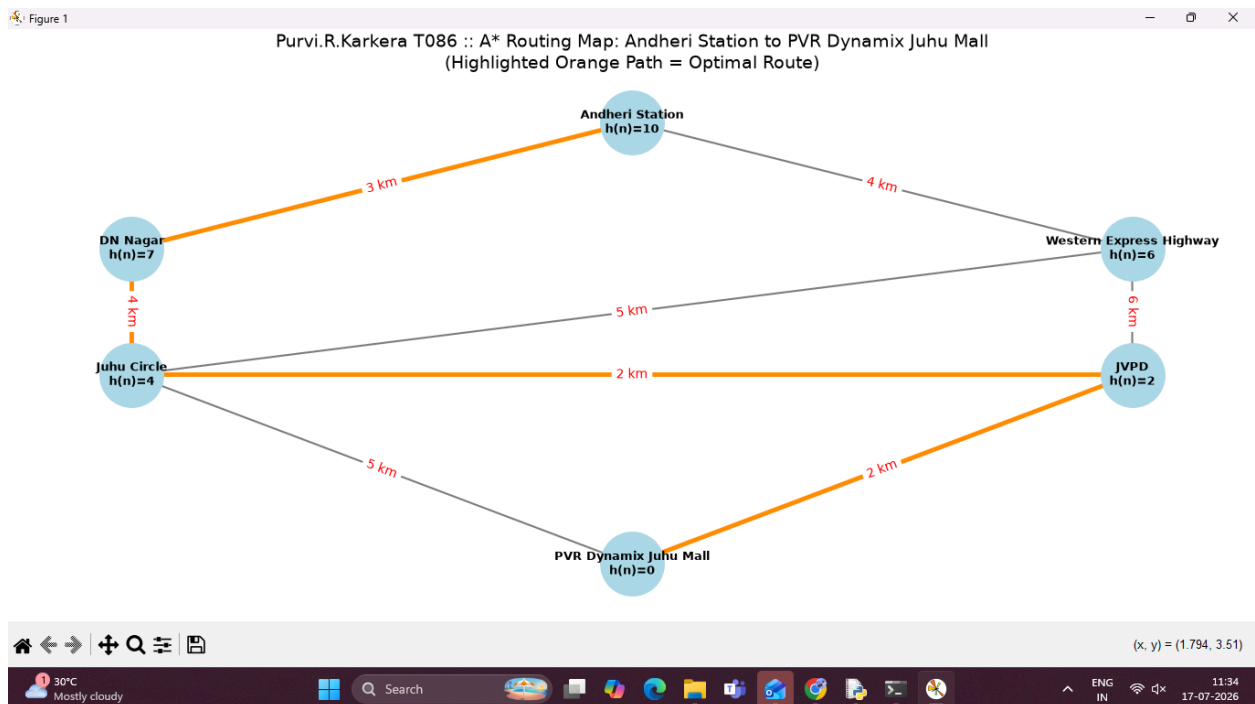
T086

Subject:Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

Output:

```
"IDLE Shell 3.14.6"
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/IT-23/Desktop/purvi T086/AI/T086 prac-2.py =====
Purvi.R.Karkera T086 :: Optimal Path Discovered: Andheri Station -> DN Nagar -> Juhu Circle -> JVPD -> PVR Dynamix Juhu Mall
Total Road Distance: 11 km
```



Purvi.R.Karkera
T086
Subject:Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

Code:Recursive Best-First Search Algorithm (RBFS)

```
import matplotlib.pyplot as plt
graph = {
    'Andheri Station': {'DN Nagar': 3, 'Western Express Highway': 4},
    'DN Nagar': {'Juhu Circle': 4},
    'Western Express Highway': {'Juhu Circle': 5, 'JVPD': 6},
    'Juhu Circle': {'JVPD': 2, 'PVR Dynamix Juhu Mall': 5},
    'JVPD': {'PVR Dynamix Juhu Mall': 2},
    'PVR Dynamix Juhu Mall': {}
}

heuristics = {
    'Andheri Station': 10,
    'DN Nagar': 7,
    'Western Express Highway': 6,
    'Juhu Circle': 4,
    'JVPD': 2,
    'PVR Dynamix Juhu Mall': 0
}

coords = {
    'Andheri Station': (2, 6),
    'DN Nagar': (1, 4.5),
    'Western Express Highway': (3, 4.5),
    'Juhu Circle': (1, 3),
    'JVPD': (3, 3),
    'PVR Dynamix Juhu Mall': (2, 1)
}
```

```
def rbfs_search(start, goal):
```

```
    success, path, cost, _ = rbfs(
        start,
        goal,
        g=0,
        f_limit=float('inf'),
        path=[start]
    )
```

Purvi.R.Karkera

T086

Subject:Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

```
return path, cost
```

```
def rbfs(node, goal, g, f_limit, path):
```

```
    if node == goal:  
        return True, path, g, g
```

```
    neighbors = graph[node]
```

```
    if not neighbors:  
        return False, [], 0, float('inf')
```

```
    successors = []
```

```
    for neighbor, distance in neighbors.items():
```

```
        if neighbor not in path:
```

```
            next_g = g + distance  
            next_f = max(  
                next_g + heuristics[neighbor],  
                g + heuristics[node]  
            )
```

```
            successors.append([next_f, neighbor, next_g])
```

```
    if not successors:  
        return False, [], 0, float('inf')
```

```
    while True:
```

```
        successors.sort(key=lambda x: x[0])
```

```
        best = successors[0]
```

```
        if best[0] > f_limit:  
            return False, [], 0, best[0]
```

```
        alternative_f = (  
            successors[1][0]  
            if len(successors) > 1
```

Purvi.R.Karkera

T086

Subject:Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

```
        else float('inf')
    )

    success, result_path, total_g, returned_f = rbfs(
        best[1],
        goal,
        best[2],
        min(f_limit, alternative_f),
        path + [best[1]]
    )

    best[0] = returned_f

    if success:
        return True, result_path, total_g, returned_f

path, total_dist = rbfs_search(
    'Andheri Station',
    'PVR Dynamix Juhu Mall'
)

print(
    f"Purvi.R.Karkera T086 :: Optimal RBFS Path: {'-> '.join(path)} ({total_dist} km)"
)

plt.figure(figsize=(8, 7))

for node, neighbors in graph.items():

    x1, y1 = coords[node]

    for neighbor, dist in neighbors.items():

        x2, y2 = coords[neighbor]

        is_path = (
            node in path and
            neighbor in path and
            path.index(neighbor) ==
            path.index(node) + 1
        )
```

Purvi.R.Karkera

T086

Subject:Artificial intelligence

```
color, width = (
    ('#2ecc71', 3)
    if is_path
    else ('#bdc3c7', 1.5)
)
```

SHETH L.U.J & SIR MV COLLEGE

```
        ha='center',
        va='center',
        color='white',
        fontsize=8,
        fontweight='bold'
    )

plt.title(
    f"Purvi.R.Karkera T086 :: RBFS Route Map: Andheri Station to PVR Dynamix Juhu Mall  

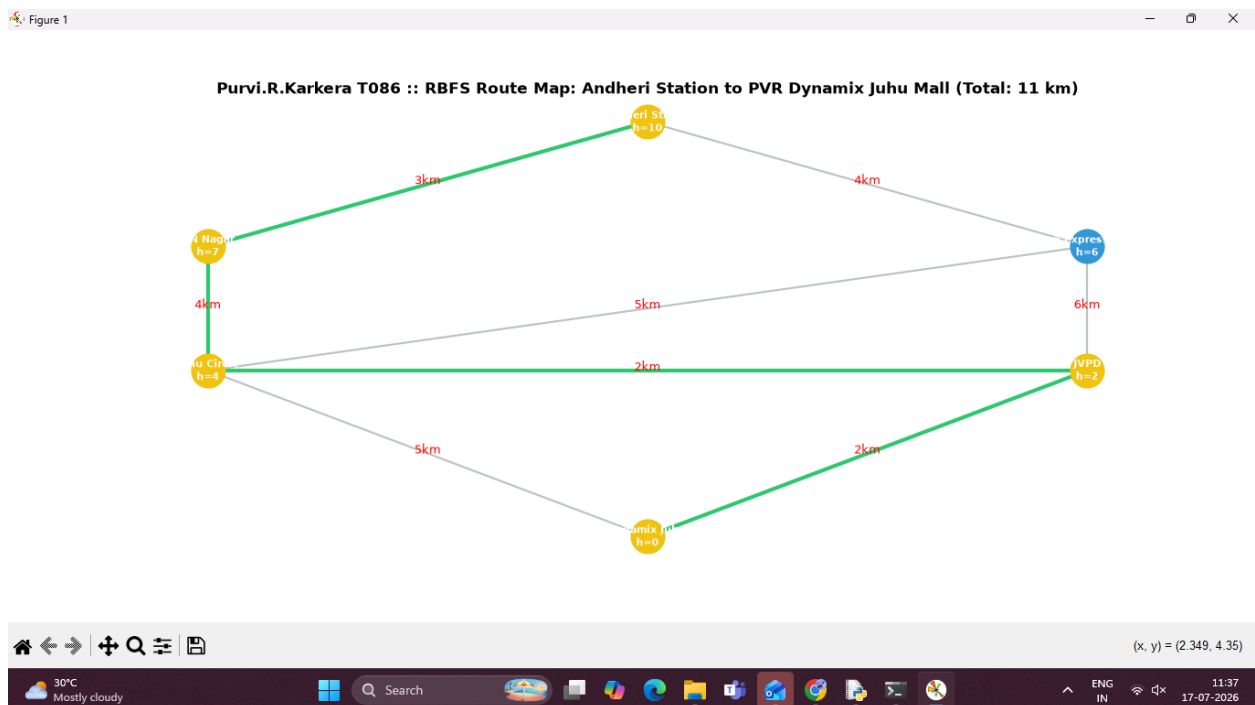
    (Total: {total_dist} km)",
    fontsize=12,
    fontweight='bold'
)

plt.axis('off')
plt.show()
```

SHETH L.U.J & SIR MV COLLEGE

Output:

```
*IDLE Shell 3.14.6*
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/IT-23/Desktop/purvi T086/AI/RBFS-T086.py =====
Purvi.R.Karkera T086 :: Optimal RBFS Path: Andheri Station -> DN Nagar -> Juhu Circle -> JVPD -> PVR Dynamix Juhu Mall (11 km)
```



Purvi.R.Karkera
T086
Subject: Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

Code:A* Search

```
print("Purvi.R.Karkera T086 :: ")
```

```
graph = {  
    'A': {'B': 1, 'C': 4},  
    'B': {'D': 2, 'E': 5},  
    'C': {'F': 1, 'G': 3},  
    'D': {'H': 4},  
    'E': {'H': 2},  
    'F': {'I': 5},  
    'G': {'I': 2},  
    'H': {'Goal': 3},  
    'I': {'Goal': 2},  
    'Goal': {}  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 5,  
    'D': 5,  
    'E': 3,  
    'F': 4,  
    'G': 2,  
    'H': 2,  
    'I': 1,  
    'Goal': 0  
}
```

```
from queue import PriorityQueue
```

```
graph = {  
    'A': {'B': 1, 'C': 4},  
    'B': {'D': 2, 'E': 5},  
    'C': {'F': 1, 'G': 3},  
    'D': {'H': 4},  
    'E': {'H': 2},  
    'F': {'I': 5},  
    'G': {'I': 2},  
    'H': {'Goal': 3},  
    'I': {'Goal': 2},
```

Purvi.R.Karkera

T086

Subject:Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

```
'Goal': {}  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 5,  
    'D': 5,  
    'E': 3,  
    'F': 4,  
    'G': 2,  
    'H': 2,  
    'I': 1,  
    'Goal': 0  
}
```

```
start = 'A'  
goal = 'Goal'
```

```
pq = PriorityQueue()  
pq.put((heuristic[start], 0, [start]))
```

```
visited = set()
```

```
while not pq.empty():
```

```
    f, g, path = pq.get()  
    node = path[-1]
```

```
    if node == goal:  
        print("Best Path:", " -> ".join(path))  
        print("Total Cost =", g)  
        break
```

```
    if node in visited:  
        continue
```

```
    visited.add(node)
```

```
    for neighbour, cost in graph[node].items():  
        new_g = g + cost  
        new_f = new_g + heuristic[neighbour]
```

Purvi.R.Karkera

T086

Subject:Artificial intelligence

SHETH L.U.J & SIR MV COLLEGE

```
print(path + [neighbour],
      "g =", new_g,
      "h =", heuristic[neighbour],
      "f =", new_f)

pq.put((new_f, new_g, path + [neighbour]))
```



Output:

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/IT-23/Desktop/purvi T086/AI/ABCD-T086.py =====
Purvi.R.Karkera T086 :
['A', 'B'] g = 1 h = 6 f = 7
['A', 'C'] g = 4 h = 5 f = 9
['A', 'B', 'D'] g = 3 h = 5 f = 8
['A', 'B', 'E'] g = 6 h = 3 f = 9
['A', 'B', 'D', 'H'] g = 7 h = 2 f = 9
['A', 'C', 'F'] g = 5 h = 4 f = 9
['A', 'C', 'G'] g = 7 h = 2 f = 9
['A', 'C', 'F', 'I'] g = 10 h = 1 f = 11
['A', 'B', 'E', 'H'] g = 8 h = 2 f = 10
['A', 'B', 'D', 'H', 'Goal'] g = 10 h = 0 f = 10
['A', 'C', 'G', 'I'] g = 9 h = 1 f = 10
['A', 'C', 'G', 'I', 'Goal'] g = 11 h = 0 f = 11
Best Path: A -> B -> D -> H -> Goal
Total Cost = 10

>>>
```